

Computer Architecture:

The Stack Frame – ISA Meeting The Compiler

Dr. Colin O'Flynn

Lectures

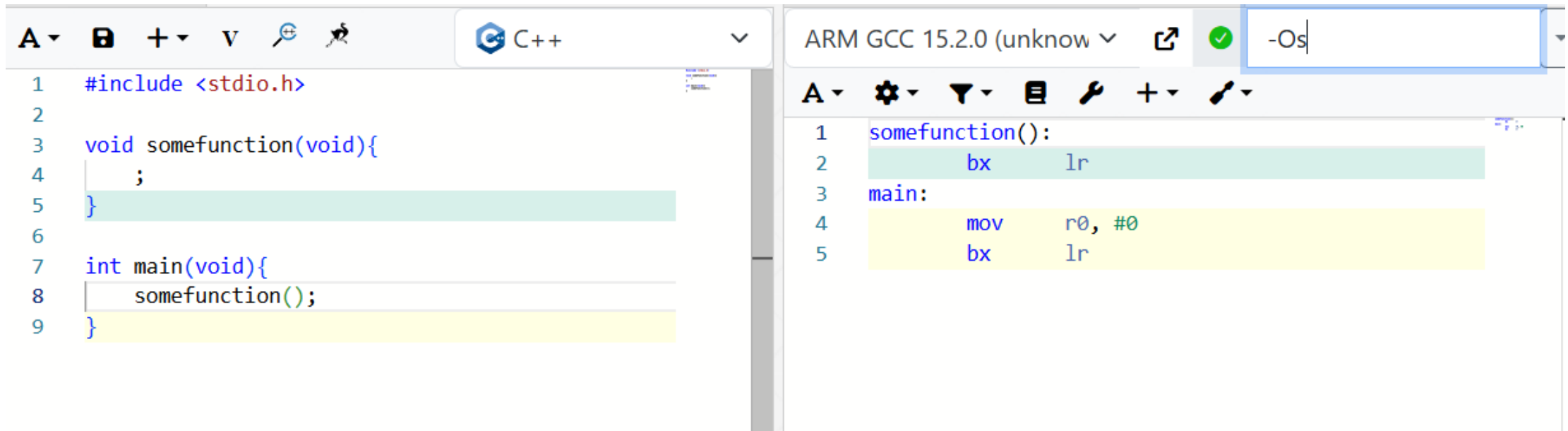
- How does a program run on a machine?
- How do you compile from high-level code to machine code?
- How do you observe the machine running?
- How do we define the machine?
- **How does the machine control flow change (planned)?**
- How does the machine control flow change (unplanned)?

...at some point: what is the *most basic possible* machine...

Link to Debuggers

- Now that you have a working debugger (or will have one shortly!), you can observe the *exact details of the machine operations*.

Calling a Function



The image shows a side-by-side comparison of C++ source code and its assembly output. The left pane is a C++ IDE with a toolbar and a dropdown menu set to 'C++'. It contains the following code:

```
1 #include <stdio.h>
2
3 void somefunction(void){
4     ;
5 }
6
7 int main(void){
8     somefunction();
9 }
```

The right pane shows the assembly output for 'ARM GCC 15.2.0 (unknown)' with the optimization flag '-Os'. It contains the following assembly code:

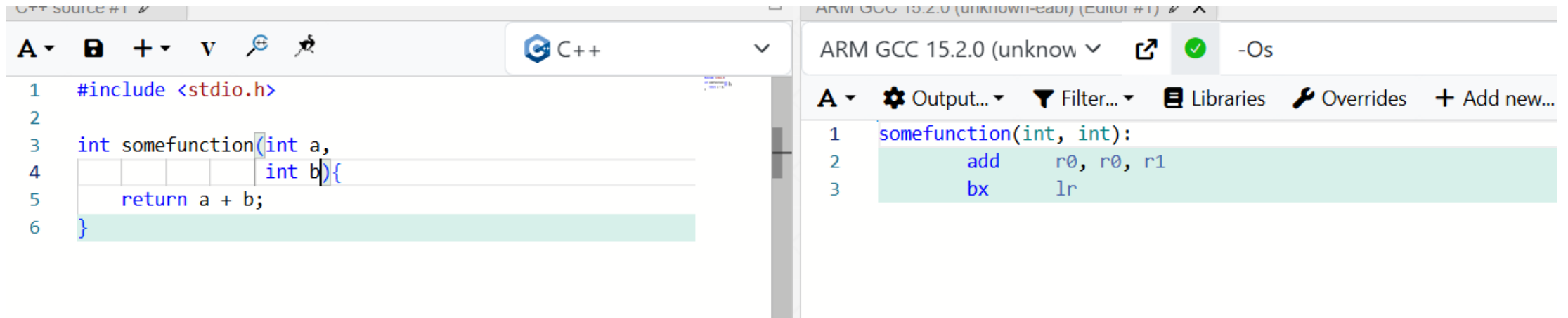
```
1 somefunction():
2     bx     lr
3 main:
4     mov    r0, #0
5     bx     lr
```

Arm Calling Notes

- LR (Link Register) = Return Address
- bl = Branch with Link (copy address to LR)

NOTE: The “lr” is not something that exists on all platforms – x86/x86-64 *does not* have this idea and stores the return address on the stack (more on that...)

Calling Conventions



The image shows a side-by-side comparison of C++ source code and its compiled ARM assembly. The left pane, titled 'C++ Source #1', shows a function 'somefunction' that takes two integers 'a' and 'b' and returns their sum. The right pane, titled 'ARM GCC 15.2.0 (unknown-eabi) (Editor #1)', shows the corresponding assembly code for the same function, using registers 'r0' and 'r1' for the arguments and 'lr' for the return value.

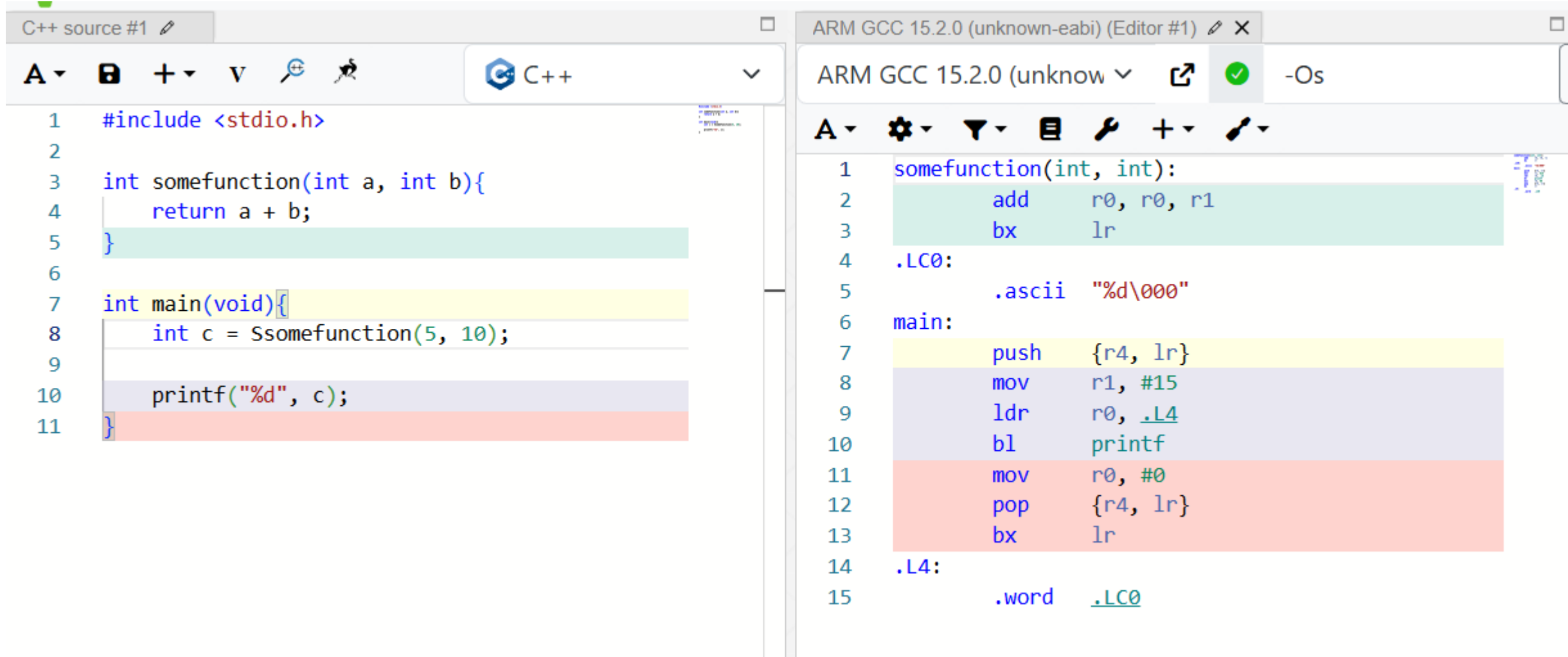
```
C++ Source #1
1 #include <stdio.h>
2
3 int somefunction(int a,
4                 int b){
5     return a + b;
6 }
```

```
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
1 somefunction(int, int):
2     add    r0, r0, r1
3     bx     lr
```

Guess so far

- Passed arguments:
 - R0 = Argument 0
 - R1 = Argument 1
- Return values:
 - R0 = Return value

Sidenote: calling conventions & -Os



```
C++ source #1
1  #include <stdio.h>
2
3  int somefunction(int a, int b){
4      return a + b;
5  }
6
7  int main(void){
8      int c = somefunction(5, 10);
9
10     printf("%d", c);
11 }
```

```
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
ARM GCC 15.2.0 (unknown-eabi) -Os
1  somefunction(int, int):
2      add    r0, r0, r1
3      bx     lr
4  .LC0:
5      .ascii "%d\000"
6  main:
7      push   {r4, lr}
8      mov    r1, #15
9      ldr    r0, .L4
10     bl     printf
11     mov     r0, #0
12     pop     {r4, lr}
13     bx     lr
14  .L4:
15     .word   .LC0
```


...what is “sp”

```
1  #include <stdio.h>
2
3  int somefunction(int a,
4                  int b,
5                  int c,
6                  int d,
7                  int e,
8                  int f){
9      return a + b + c + d + e + f;
10 }
```

```
ARM GCC 15.2.0 (unknown) -Os
1  somefunction(int, int, int, int, int, int):
2      add    r0, r0, r1
3      add    r0, r0, r2
4      add    r0, r0, r3
5      ldr    r3, [sp]
6      add    r0, r0, r3
7      ldr    r3, [sp, #4]
8      add    r0, r0, r3
9      bx     lr
```

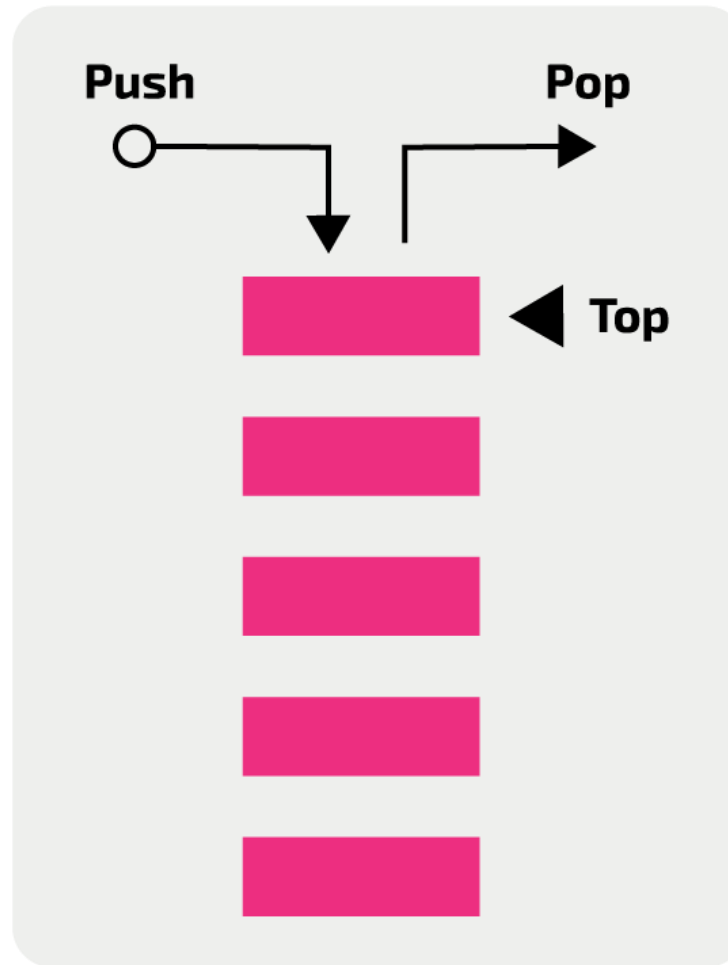
The Stack

```
C++ source #1
1 #include <stdio.h>
2
3 int somefunction(int a, int b){
4     return a + b;
5 }
6
7 int main(void){
8     int c = somefunction(5, 10);
9
10    printf("%d", c);
11 }
```

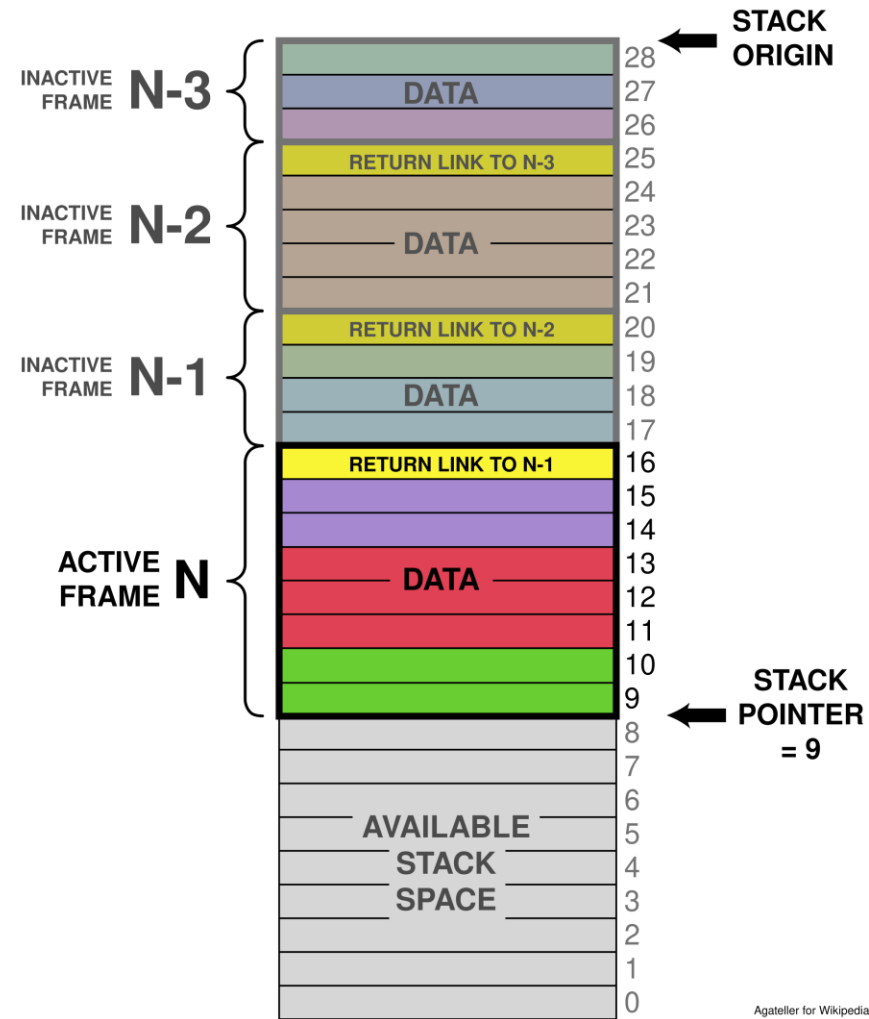
```
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
ARM GCC 15.2.0 (unknown-eabi)
-O0

1 somefunction(int, int):
2     str    fp, [sp, #-4]!
3     add    fp, sp, #0
4     sub    sp, sp, #12
5     str    r0, [fp, #-8]
6     str    r1, [fp, #-12]
7     ldr    r2, [fp, #-8]
8     ldr    r3, [fp, #-12]
9     add    r3, r2, r3
10    mov    r0, r3
11    add    sp, fp, #0
12    ldr    fp, [sp], #4
13    bx     lr
14
15 .LC0:
16     .ascii "%d\000"
17
18 main:
19     push   {fp, lr}
20     add    fp, sp, #4
21     sub    sp, sp, #8
22     mov    r1, #10
23     mov    r0, #5
24     bl     somefunction(int, int)
25     str    r0, [fp, #-8]
26     ldr    r1, [fp, #-8]
27     ldr    r0, .L5
28     bl     printf
29     mov    r3, #0
30     mov    r0, r3
31     sub    sp, fp, #4
32     pop    {fp, lr}
33     bx     lr
34
35 .L5:
36     .word  .LC0
```

The Stack



The Stack



The Stack

IMPORTANT: Stack is just memory (RAM). We give it a special name but nothing stops you from read/writing to the stack.

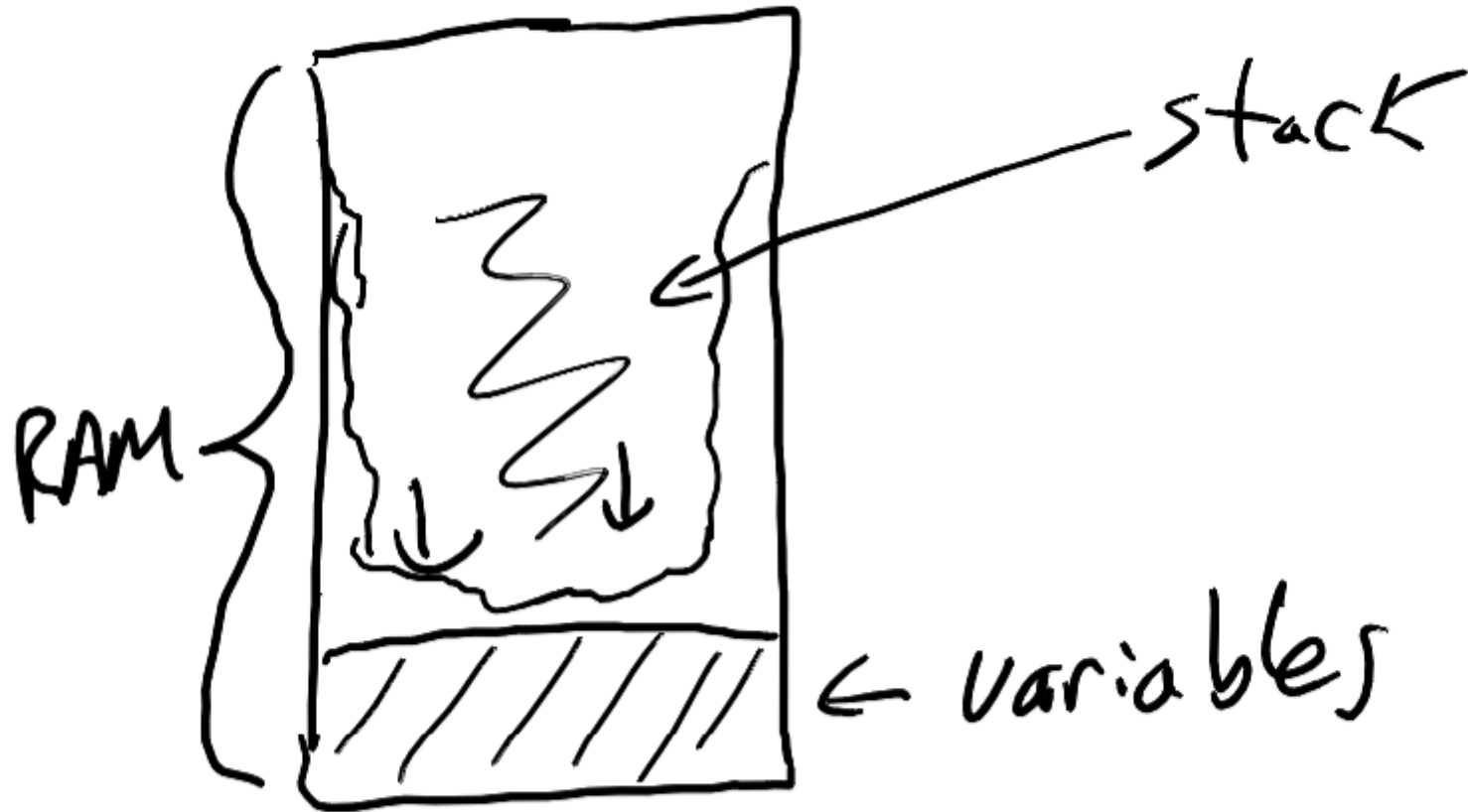
...also nothing stops the stack space from read/writing into memory...

Stack Growth

- Stack normally grows downward on newer embedded platforms (Arm/AVR), but check on others

Stack & Variable Storage?

- Stack also stores local variables.
- **WARNING:** Stack can collide with memory – this results in memory corruption and other issues.
- **CHECKING STACK SIZE:** You can write known patterns to memory & inspect how far “overwritten” they got:



Prologue & Epilogue

```
C++ source #1
1 #include <stdio.h>
2
3 int somefunction(int a, int b){
4     return a + b;
5 }
6
7 int main(void){
8     int c = somefunction(5, 10);
9
10    printf("%d", c);
11 }
```

```
ARM GCC 15.2.0 (unknown-eabi) (Editor #1)
ARM GCC 15.2.0 (unknown-  -O0

1 somefunction(int, int):
2     str    fp, [sp, #-4]!
3     add    fp, sp, #0
4     sub    sp, sp, #12
5     str    r0, [fp, #-8]
6     str    r1, [fp, #-12]
7
8     ldr    r2, [fp, #-8]
9     ldr    r3, [fp, #-12]
10    add    r3, r2, r3
11
12    mov    r0, r3
13    add    sp, fp, #0
14    ldr    fp, [sp], #4
15    bx     lr
16
17 .LC0:
18     .ascii "%d\000"
19
20 main:
21    push    {fp, lr}
22    add    fp, sp, #4
23    sub    sp, sp, #8
24    mov    r1, #10
25    mov    r0, #5
26    bl     somefunction(int, int).
27    str    r0, [fp, #-8]
28    ldr    r1, [fp, #-8]
29    ldr    r0, .L5
30    bl     printf
31    mov    r3, #0
32    mov    r0, r3
33    sub    sp, fp, #4
34    pop    {fp, lr}
35    bx     lr
36
37 .L5:
38     .word  .LC0
```

Prologue

Epilogue

Copy/Paste Slide (Ignore)

```
#include <stdio.h>

int somefunction(int a,
                 int b){
    int i,j,k,l,m,n,o,p;

    for (int i = 0; i < a; i++){
        k++;
        k += b;
        l += 3;
        m += 4;
        n += 5;
        o += 6;
        p += 7;
    }

    return a + b + k + l + m + n + n + o + p;
}
```

Caller-saved Registers

The image displays a side-by-side comparison of C++ source code and its corresponding ARM assembly output, generated by ARM GCC 15.2.0 (unknown-eabi).

C++ Source Code (Left Panel):

```
1 #include <stdio.h>
2
3 int somefunction(int a,
4                 int b){
5     int i,j,k,l,m,n,o,p;
6
7     for (int i = 0; i < a; i++){
8         k++;
9         k += b;
10        l += 3;
11        m += 4;
12        n += 5;
13        o += 6;
14        p += 7;
15    }
16
17    return a + b + k + l + m + n + n + o + p;
18 }
```

ARM Assembly Code (Right Panel):

```
1 somefunction(int, int):
2     bic    r2, r0, r0, asr #31
3     push   {r4, r5, lr}
4     add    r3, r0, r1
5     mla    r5, r1, r2, r2
6     mov    r1, #6
7     add    r4, r2, r2, lsl #1
8     add    r3, r3, r5
9     add    r3, r3, r4
10    add    lr, r2, r2, lsl #2
11    add    r3, r3, r2, lsl #2
12    add    r3, r3, lr, lsl #1
13    mla    r3, r1, r2, r3
14    rsb    ip, r2, r2, lsl #3
15    add    r0, r3, ip
16    pop     {r4, r5, lr}
17    bx     lr
```

The assembly code shows the function `somefunction` taking two arguments. The registers `r4`, `r5`, and `lr` are pushed onto the stack at the start of the function (line 3) and popped at the end (line 16). The `lr` register is used as a pointer to the next instruction, and the `bx` instruction is used to branch to it (line 17).

Callee-saved Registers

```
1  #include <stdio.h>
2
3  int somefunction(int a,
4                  int b){
5      int i,j,k,l,m,n,o,p;
6
7      a = somefunction(a, b);
8
9      for (int i = 0; i < a; i++){
10         k++;
11         k += b;
12         l += 3;
13         m += 4;
14         n += 5;
15         o += 6;
16         p += 7;
17     }
18
19     return a + b + k + l + m + n + n + o + p;
20
21 }
22
```

A ▾ ⚙ Output... ▾ 🔍 Filter... ▾ 📖 Libraries 🔑 Overrides + Add new... ▾

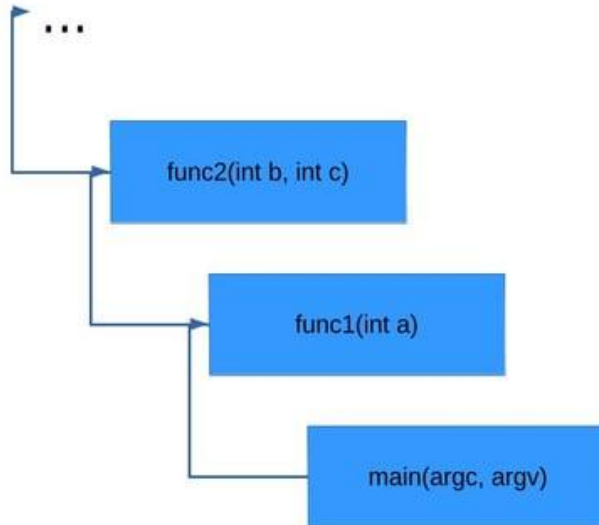
```
1  somefunction(int, int):
2      push    {r4, r5, r6, lr}
3      mov     r4, r1
4      bl      somefunction(int, int)
5      bic     r2, r0, r0, asr #31
6      add     r3, r4, r0
7      mov     r0, #6
8      mla     r5, r4, r2, r2
9      add     lr, r2, r2, lsl #1
10     add     r3, r3, r5
11     add     r3, r3, lr
12     add     ip, r2, r2, lsl #2
13     add     r3, r3, r2, lsl #2
14     add     r3, r3, ip, lsl #1
15     mla     r0, r2, r0, r3
16     rsb     r1, r2, r2, lsl #3
17     add     r0, r0, r1
18     pop     {r4, r5, r6, lr}
19     bx      lr
```

ARM Cortex-M Calling Conventions & Registers

- **Arguments:**
 - **First 4:** r0 – r3
 - **Remaining:** stack
- **Return value** in r0
- **Registers:**
 - Caller-saved: r0-r3, r12
 - *HINT: r0-r3... see above*
 - Callee-saved r4-r11
- **Link Register (LR)**
 - **R14**
 - May be reused as a “regular register”
- **Intra-procedure-call scratch register (IP)**
 - **R12**
 - May be reused as a “regular register”

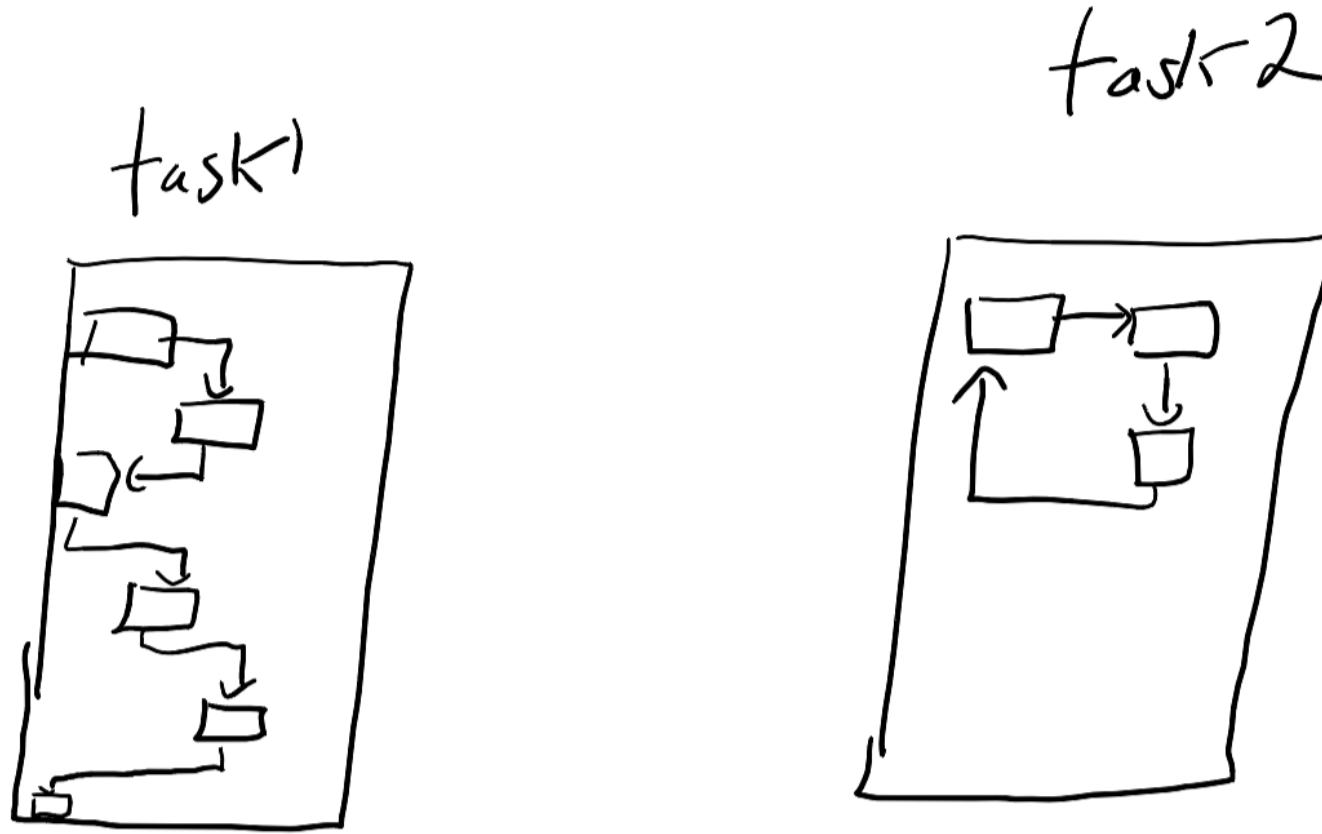
Stack Trace & Debuggers

```
#0 func3 () at main.c:2  
#1 0x000055555555150 in func2 (c=7, b=8) at main.c:7  
#2 0x000055555555171 in func1 (a=5) at main.c:11  
#3 0x000055555555191 in main (argc=1, argv=0x7fffffffe138) at main.c:15
```



Remember: each of these is a *Stack Frame*

Stack Frames & Context Switching



Context Switching

- We can *save* and *restore* the system context:
 - All registers
 - Naturally saves & restores stack pointer
 - Requires us to setup memory segments so these do not overlap

Lab & Assignment Explorations

- Lab #1 is about debugger usage – but you can also take a look at stack frames in the debugger.
- You'll be asked to look in more detail of these stack frames in later Labs & assignments, but I suggest taking a look at it in your first lab to get an understanding for the tools.

Stack Summary

- The *Stack* is memory used by functions:
 - Arguments
 - Local storage
 - Return address (sometimes)
- On most systems nothing stops *stack overrun*
- The *Stack Frame* is a specific section of memory used by each function.
 - We can trace through Stack Frames to understand call order (“stack trace”)
- Callee vs. Caller-Saved
 - Critical to understand who is responsible for saving register state
 - Caller-saved registers normally used for argument passing anyway